УИР 1 «Кодирование данных»

Полный конспект теории для защиты

Данилов Т. Н., P3331, ТКС, преподаватель Алиев Т. И.

Сообщение ДаТН = `C4 E0 D2 CD`, $C = 10$ Мбит/с

Содержание

1. Как пользоваться конспектом . . . 2
2. Основные понятия . . . 2
   2.1. Сигнал, бит, бод . . . 2
   2.2. Спектр, гармоники, полоса пропускания . . . 2
   2.3. Что требуется от хорошего кода . . . 3
3. Физическое кодирование . . . 3
   3.1. NRZ (потенциальный, без возврата к нулю) . . . 3
   3.2. NRZI . . . 3
   3.3. RZ (биполярный импульсный, с возвратом к нулю) . . . 3
   3.4. AMI (биполярный, с чередованием полярности) . . . 4
   3.5. MLT-3 . . . 4
   3.6. Манчестерский код . . . 4
   3.7. Дифференциальный манчестерский (на случай вопроса) . . . 4
   3.8. PAM-5 (на случай вопроса «наименьшая верхняя граница») . . . 4
   3.9. Сводная таблица кодов (теоретические границы при скорости C) . . . 5
4. Методика расчёта частот (то, что вы делали в отчёте) . . . 5
   4.1. Правило . . . 5
   4.2. Что считать полуволной в каждом коде (по вашему отчёту) . . . 6
   4.3. Разбор NRZ на вашем сообщении (его спросят первым) . . . 6
   4.4. Пример на байте C5 = 1100 0101 (его любят просить нарисовать) . . . 6
5. Логическое кодирование . . . 7
   5.1. Избыточное кодирование . . . 7
   5.2. Скремблирование . . . 8
   5.3. Сравнение . . . 8
6. Ваш отчёт: числа и что говорить . . . 9
   6.1. Сообщение и физические коды (стр. 3–8) . . . 9
   6.2. Логическое кодирование и итог (стр. 8–13) . . . 9
7. Слабое место отчёта: краевые импульсы MLT-3 . . . 10
8. Вопросы на защиту с ответами . . . 11
9. Шпаргалка . . . 13

1. Как пользоваться конспектом

1. Разделы 2–4: общая теория (она нужна для контрольных вопросов).
2. Раздел 5: ваш отчёт, число за числом, и «что говорить».
3. Раздел 6: слабое место отчёта (краевые импульсы MLT-3). Прочитайте обязательно.
4. Раздел 7: вопросы с ответами. Раздел 8: шпаргалка на одну страницу.

Всё, что преподаватель может спросить, сводится к трём вещам: **(1)** как по диаграмме найти длины полуволн и частоты, **(2)** почему у кода такие достоинства и недостатки (синхронизация, постоянная составляющая, ширина спектра, число уровней), **(3)** зачем логическое кодирование и чем 4B/5B отличается от скремблирования.

2. Основные понятия

2.1. Сигнал, бит, бод

- **Битовый интервал** $\tau = \frac{1}{C}$. При $C = 10$ Мбит/с: $\tau = 0,1$ мкс = 100 нс.
- **Бит/с (битовая скорость)** — сколько информационных бит передаётся за секунду.
- **Бод (скорость модуляции)** — сколько раз в секунду может меняться состояние сигнала (символов/с). Бод равен бит/с, если на каждый бит приходится один символ (NRZ, NRZI, AMI, MLT-3). У манчестерского кода и RZ на бит приходится два состояния, поэтому бод $= 2C$. У кода с несколькими битами на символ (PAM-5: 2 бита на символ) бод меньше бит/с.
- **Пропускная способность (по Шеннону)**: $C_{\max} = F \log_2(1 + S/N)$. **По Найквисту** (без шума): $C = 2F \log_2 M$, где M — число уровней сигнала. Больше уровней при той же полосе даёт больше бит/с, но хуже помехоустойчивость.

2.2. Спектр, гармоники, полоса пропускания

- Любой периодический сигнал раскладывается в сумму синусоид (гармоник) — ряд Фурье.
- **Прямоугольный сигнал** вида 1010… с частотой повторения f_0 состоит из гармоник $f_0, 3f_0, 5f_0, 7f_0, \ldots$ с амплитудами $A, A/3, A/5, A/7, \ldots$. Основная гармоника несёт главную информацию, остальные «делают углы» фронтов.
- Для 1010… в NRZ один период — это два бита (1 и 0), поэтому $f_0 = C/2$.

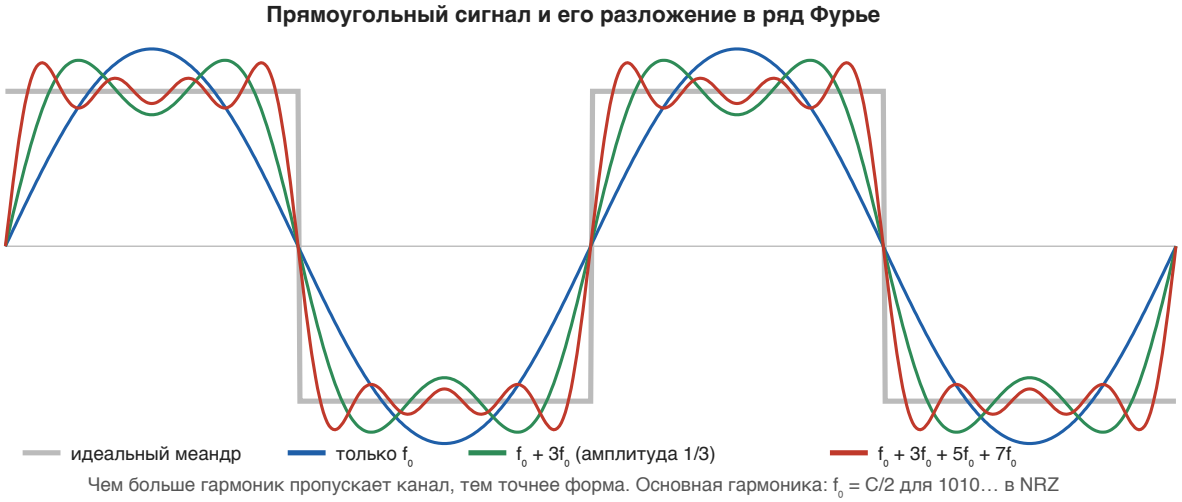


Рисунок 1. Чем больше гармоник проходит через канал, тем ближе сигнал к идеальному прямоугольнику

- **Спектр сигнала** — диапазон частот его гармоник. В работе берётся диапазон основных гармоник: от $f_{\text{н}}$ до $f_{\text{в}}$, ширина $S = f_{\text{в}} - f_{\text{н}}$.
- **Полоса пропускания канала** F — диапазон частот, который канал передаёт с ослаблением не более 3 дБ (амплитуда не ниже $1/\sqrt{2}$). Условие качественной передачи: **канал должен пропускать частоты от $f_{\text{н}}$ до $f_{\text{в}}$ и $F \geq S$**.
- Чем шире спектр кода, тем дороже и хуже канал нужен. «Меньше спектр — выше скорость в той же полосе».

2.3. Что требуется от хорошего кода

1. Узкий спектр (малая $f_{\text{в}}$, высокая $f_{\text{н}}$).
2. Самосинхронизация: сигнал достаточно часто меняется, и приёмник подстраивает часы.
3. Нет постоянной составляющей (её не пропускают трансформаторы гальванической развязки и каналы без постоянного тока).
4. Обнаружение ошибок.
5. Малое число уровней (проще и дешевле).

Синхронизация. Приёмник снимает отсчёт в середине битового интервала. Часы передатчика и приёмника расходятся, и на длинной серии одинаковых бит без смен уровня приёмник теряет границы битов (пропускает бит или считывает дважды).

Постоянная составляющая появляется, когда время на одном уровне заметно больше, чем на другом (длинные серии нулей или единиц в NRZ, импульсы одной полярности в RZ). Низкие частоты $\to f_{\text{н}} \to 0$.

3. Физическое кодирование

Физическое кодирование — как биты превращаются в сигнал на линии. Коды делят на **потенциальные** (бит задаётся уровнем на всём интервале: NRZ, NRZI, AMI, MLT-3) и **импульсные** (бит задаётся импульсом или перепадом внутри интервала: RZ, манчестерский). У импульсных кодов спектр шире, зато есть самосинхронизация.

3.1. NRZ (потенциальный, без возврата к нулю)

- 1 — высокий уровень, 0 — низкий, уровень не меняется внутри бита. Два уровня.
- Плюсы: простейший, самый узкий спектр среди двухуровневых ($f_{\text{в}} = C/2$).
- Минусы: нет самосинхронизации, постоянная составляющая, на длинных сериях $f_{\text{н}} \to 0$, нет контроля ошибок.

3.2. NRZI

- Двухуровневый. На единице уровень **инвертируется**, на нуле сохраняется (в отчёте не используется, но в вопросах бывает).
- Длинная серия единиц даёт меандр (синхронизация есть), длинная серия нулей — постоянный уровень (проблема).

3.3. RZ (биполярный импульсный, с возвратом к нулю)

- 1 — положительный импульс, 0 — отрицательный. Импульс занимает первую половину бита, во второй половине сигнал на нуле. Три уровня.
- Плюсы: самосинхронизация (в каждом бите есть возврат к нулю).
- Минусы: три уровня, широкий спектр (до C), на сериях одинаковых бит — импульсы одной полярности, то есть постоянная составляющая.

- Бод $= 2C$.

3.4. AMI (биполярный, с чередованием полярности)

- 0 — нулевой потенциал, 1 — попеременно $+$ и $-$. Три уровня.
- Плюсы: нет постоянной составляющей на единицах, **нарушение чередования полярности = ошибка** (контроль ошибок).
- Минусы: на сериях нулей линия стоит на нуле (нет синхронизации), три уровня.

3.5. MLT-3

- Три уровня. На единице уровень переходит к следующему по циклу $0 \to + \to 0 \to - \to 0$, на нуле сохраняется.
- Длинная серия единиц даёт период в 4 бита, поэтому **максимальная основная гармоника $C/4$** (у NRZ — $C/2$). Самый узкий спектр из рассмотренных. Применяется в 100BASE-TX вместе с 4B/5B.
- Минусы: три уровня, на сериях нулей нет смены уровня (поэтому нужно логическое кодирование).
- Постоянной составляющей в среднем нет, потому что уровни $+$ и $-$ чередуются.
- Бод $= C$.

3.6. Манчестерский код

- В середине каждого бита обязательно есть перепад. В отчёте: 1 — перепад от высокого к низкому, 0 — от низкого к высокому. Между одинаковыми соседними битами на границе интервала добавляется служебный перепад.

  > В стандарте IEEE 802.3 (Ethernet 10BASE-T) наоборот: 1 — перепад от низкого к высокому. Если спросят — «в моём задании по таблице преподавателя 1 = сверху вниз, в Ethernet принято наоборот; это условность, на частоты не влияет».

- Плюсы: самосинхронизация в каждом бите, нет постоянной составляющей, два уровня. $f_{\text{н}}$ не ниже $C/2$ при любых данных.
- Минусы: самый широкий спектр, $f_{\text{в}} = C$. Бод $= 2C$.

3.7. Дифференциальный манчестерский (на случай вопроса)

- Перепад в середине бита есть всегда (синхронизация). Бит задаётся наличием перепада на **границе** интервала: 0 — есть, 1 — нет (в варианте Token Ring).

3.8. PAM-5 (на случай вопроса «наименьшая верхняя граница»)

- 5 уровней, 2 бита на символ, $f_{\text{в}} = C/4$ по символьной скорости $C/2$ бод. Применяется в 1000BASE-T. Самый узкий спектр, но самые сложные приёмник и передатчик.

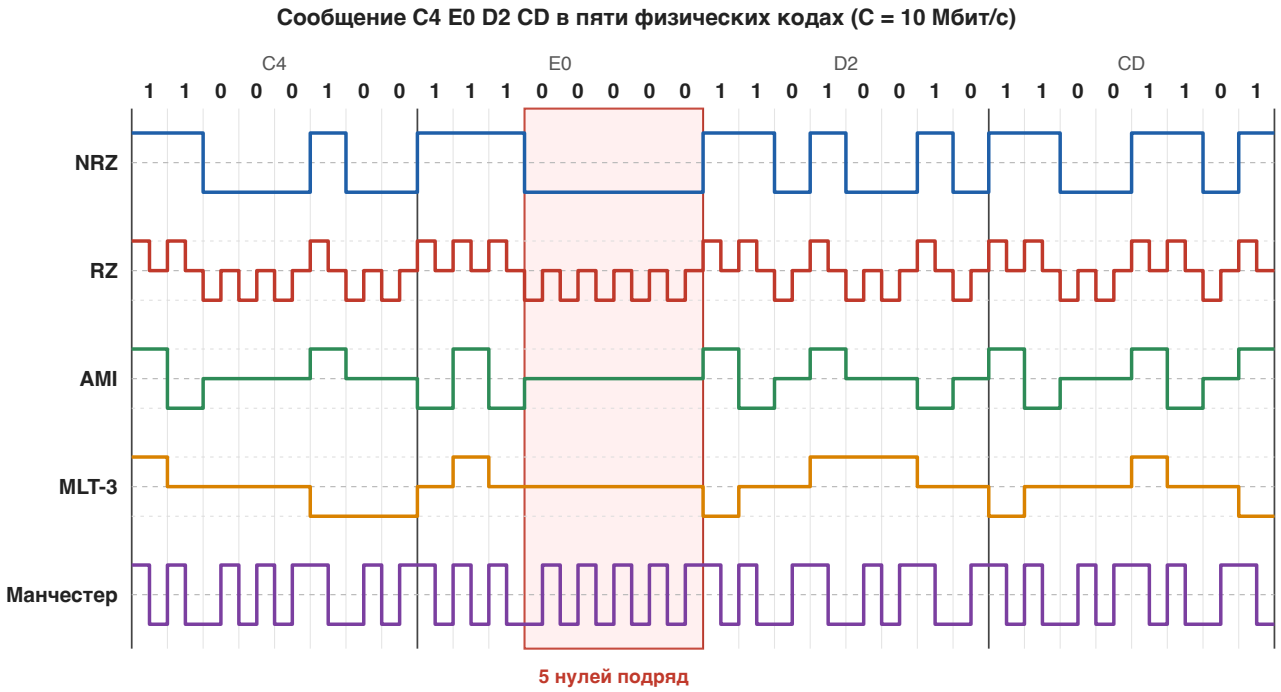


Рисунок 2. Все пять физических кодов для сообщения ДаТН. Розовым выделена серия из 5 нулей: она определяет $f_{\text{н}}$

3.9. Сводная таблица кодов (теоретические границы при скорости C)

Код	Уровни	Самосинхр.	Пост. составл.	$f_{\text{в}}$	Контроль ошибок
NRZ	2	нет	есть на сериях	$C/2$	нет
RZ	3	да	на сериях одинаковых бит	C	нет
AMI	3	нет на нулях	нет на единицах	$C/2$	да (чередование)
MLT-3	3	нет на нулях	нет	$C/4$	нет
Манчестер	2	да	нет	C	нет

4. Методика расчёта частот (то, что вы делали в отчёте)

4.1. Правило

Сигнал разбивается на участки. Каждый участок — **полуволна** основной гармоники. Если полуволна длится L битовых интервалов, то период гармоники $T = 2L\tau$, и

$$f = \frac{1}{2L\tau} = \frac{C}{2L}.$$

Почему именно так: за один период синусоида проходит «плюс» и «минус», то есть две полуволны. Участок постоянного уровня длиной $L\tau$ — это одна полуволна, значит период вдвое длиннее. Пример: чередование 1010 в NRZ, $L = 1$, $f = C/2 = 5$ МГц.

Границы:

- $f_{\text{н}}$ — частота самой длинной полуволны (минимальная частота);
- $f_{\text{в}}$ — частота самой короткой полуволны (максимальная частота);
- $S = f_{\text{в}} - f_{\text{н}}$;
- $f_{\text{ср}} = (\sum f_i n_i)/(\sum n_i)$ — среднее арифметическое частот всех полуволн (в отчёте не взвешено по времени);

- $F \geq S$, и канал должен пропускать диапазон $[f_н, f_в]$.

4.2. Что считать полуволной в каждом коде (по вашему отчёту)

- **Двухуровневые (NRZ, манчестер):** полуволна — участок постоянного уровня.
- **Трёхуровневые (AMI, MLT-3):** полуволна — импульс одной полярности. Нулевой участок между импульсами **делится пополам** между соседними импульсами (синусоида посередине нулевого участка проходит через ноль).
- **RZ:** соседние одинаковые биты дают шаблон $+0+0$ с периодом $T = \tau$, $f = C$; разные биты дают $+0-0$ с периодом 2τ, $f = C/2$. Считаются 31 стык между 32 битами.
- **Манчестер:** уровень держится либо полбита (там, где на границе служебный перепад): $f = C$; либо целый бит (стык разных бит): $f = C/2$.

4.3. Разбор NRZ на вашем сообщении (его спросят первым)

Биты: `1100 0100 1110 0000 1101 0010 1100 1101`. Участки: `11|000|1|00|111|00000|11|0|1|00|1|0|11|00|11|0|1` — 17 штук.

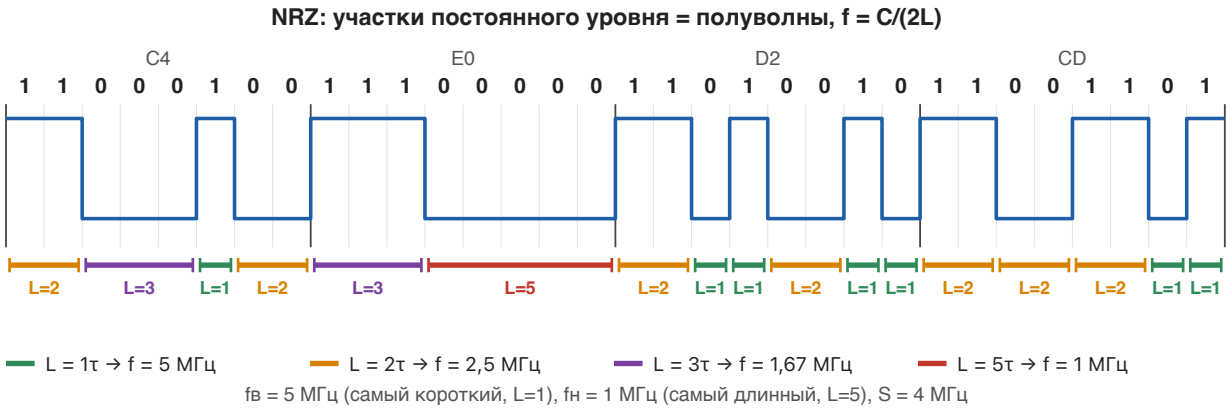


Рисунок 3. Полуволны NRZ: чем длиннее участок, тем ниже частота

Длина L, τ	$f = C/(2L)$, МГц	Сколько n_i	$f_i n_i$
1	5	7	35
2	2,5	7	17,5
3	1,67	2	3,33
5	1	1	1
Итого		17	56,83

$f_н = 1$ МГц (пять нулей на стыке E0 и D2), $f_в = 5$ МГц, $S = 4$ МГц, $f_{ср} = 56,83/17 \approx 3,34$ МГц.

4.4. Пример на байте C5 = 1100 0101 (его любят просить нарисовать)

```
биты:   1   1   0   0   0   1   0   1
NRZ:    +   +   −   −   −   +   −   +
RZ:     +0  +0  −0  −0  −0  +0  −0  +0   (импульс в первой половине бита)
AMI:    +   −   0   0   0   +   0   −
NRZI:   +   −   −   −   −   +   +   −    (старт с низкого, инверсия на 1)
MLT-3:  +   0   0   0   0   −   −   0    (старт с 0, цикл 0,+,0,−)
Манч.:  +−  +−  −+  −+  −+  +−  −+  +−   (1: сверху вниз, 0: снизу вверх)
```

5. Логическое кодирование

Логическое кодирование выполняется **до** физического: оно меняет сами биты, чтобы у потенциальных кодов (NRZ, AMI, MLT-3) не было длинных серий нулей или единиц. Цель: самосинхронизация, отсутствие постоянной составляющей, сужение спектра снизу, иногда — контроль ошибок. Два метода: **избыточное кодирование** и **скремблирование**.

5.1. Избыточное кодирование

Группа из m бит заменяется символом из $n > m$ бит по таблице. Из 2^n символов выбирают 2^m «хороших» (без длинных нулей и с нужной балансировкой), остальные — **запрещённые**. Запрещённый код на приёме = ошибка.

Избыточность $= (n - m)/m$. Недостаток: линия должна работать быстрее. Чтобы полезная скорость осталась C, скорость в линии $C' = C \cdot n/m$ (для 4B/5B: 12,5 Мбит/с), поэтому растут все частоты и расширяется спектр.

Метод	Избыточность	Запрещённых кодов	Применение	Примечание
4B/5B	25 %	$32 - 16 = 16$	100BASE-TX, FDDI	не больше 3 нулей подряд
5B/6B	20 %	$64 - 32 = 32$	100VG-AnyLAN	
8B/10B	25 %	$1024 - 256 = 768$	Gigabit Ethernet (волокно), Fibre Channel, SATA	баланс единиц и нулей
8B/6T	3^6 = 729 на 256	$729 - 256 = 473$	100BASE-T4	8 бит в 6 троичных символов

Таблица 4B/5B (вся, на случай вопроса «закодируйте байт»):

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
11110	01001	10100	10101	01010	01011	01110	01111	10010	10011	10110	10111	11010	11011	11100	11101

Служебные символы (для общего развития): I `11111` (простой), J `11000` и K `10001` (начало кадра), T `01101` (конец), R `00111`, S `11001`, Q `00000`, H `00100`.

Почему не больше трёх нулей подряд: у каждого кода в таблице не больше одного нуля в начале и не больше двух нулей в конце, поэтому на стыке двух кодов максимум $2 + 1 = 3$ нуля.

Ваш пример: C4 E0 D2 CD $\to$ C=`11010`, 4=`01010`, E=`11100`, 0=`11110`, D=`11011`, 2=`10100`, C=`11010`, D=`11011`. Всего 40 бит: `D2 B9 ED D3 5B`. Серии: не более 2 нулей и 4 единиц. Линейная скорость $C' = 1,25 \cdot 10 = 12,5$ Мбит/с, $\tau' = 0,08$ мкс.

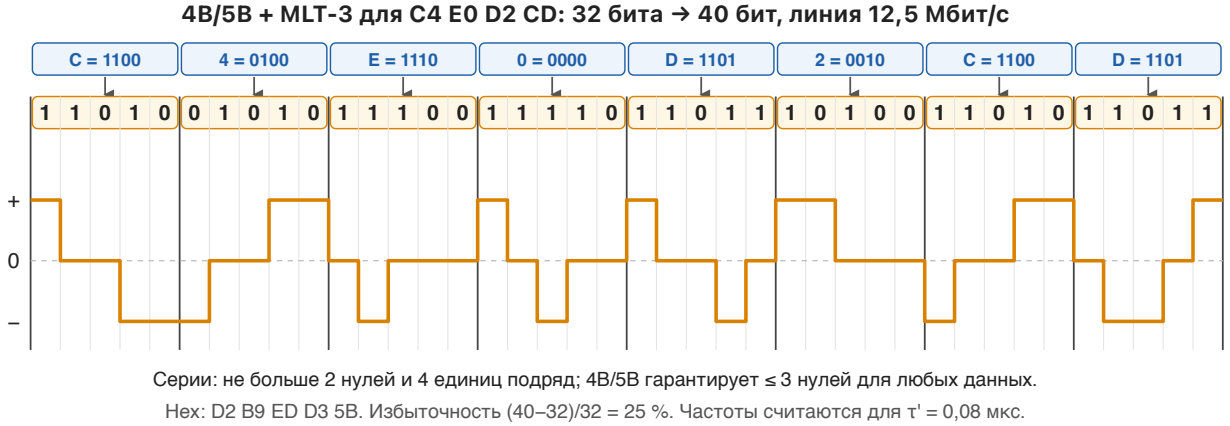


Рисунок 4. Конвейер 4B/5B + MLT-3 на вашем сообщении

5.2. Скремблирование

- Каждый бит исходного потока смешивается по XOR с предыдущими битами **результата**:

$$B_i = A_i \oplus B_{i-j} \oplus B_{i-k}.$$

  Длина не меняется (избыточности нет). Разряды с номером меньше 1 считаются нулевыми (в отчёте).
- **Дескремблер:** $C_i = B_i \oplus B_{i-j} \oplus B_{i-k} = A_i$ (формула та же, но на приёмнике берутся принятые биты).
- Полиномы задания: $B_i = A_i \oplus B_{i-3} \oplus B_{i-5}$ и $B_i = A_i \oplus B_{i-5} \oplus B_{i-7}$.
- **Первые шаги** для $B_i = A_i \oplus B_{i-5} \oplus B_{i-7}$ и A = `1100 0100` …: $B_1 = A_1 = 1$, $B_2 = 1$, $B_3 = B_4 = B_5 = 0$, $B_6 = A_6 \oplus B_1 = 1 \oplus 1 = 0$, $B_7 = A_7 \oplus B_2 = 0 \oplus 1 = 1$, $B_8 = A_8 \oplus B_3 \oplus B_1 = 0 \oplus 0 \oplus 1 = 1$. Итог по всему сообщению: `C3 7D C7 78` (19 единиц, серия нулей 4, серия единиц 5).
- Для полинома $B_{i-3} \oplus B_{i-5}$ итог: `D9 09 AA C3` (15 единиц, серия нулей 4, серия единиц 2).

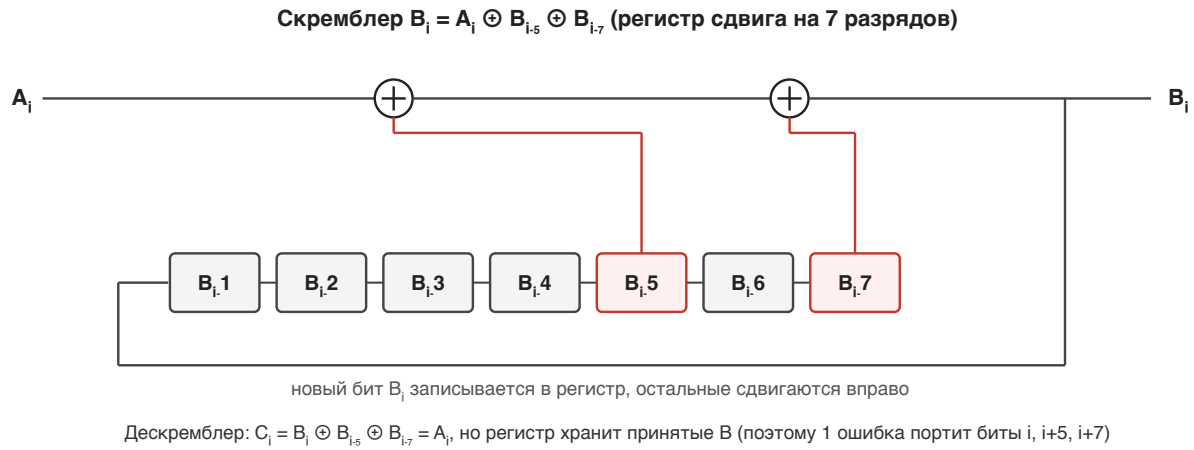


Рисунок 5. Схема скремблера и дескремблера для полинома 5/7

Свойства и недостатки:

- Не гарантирует отсутствия длинных серий: результат зависит от данных. Из-за этого в отчёте серия нулей сократилась только с 5 до 4.
- **Размножение ошибок:** один искажённый бит B_i портит три бита на выходе дескремблера: i, $i + j$, $i + k$.
- Скремблер и дескремблер должны быть синхронизированы по начальному состоянию.
- Поток из одних нулей при нулевом начальном состоянии остаётся нулевым ($0 \oplus 0 \oplus 0 = 0$). При единичном начальном состоянии и примитивном полиноме получается псевдослучайная последовательность.
- Скремблер взаимно однозначно переставляет сообщения одной длины, поэтому в среднем по всем входам он не лучше случайных бит (у случайных бит около 15,6 % бит попадает в серии нулей длиной от 4). Полезен там, где данные хуже случайных (длинные серии одинаковых бит).

Инверсия ($B_i = A_i \oplus 1$) — самый простой приём: на вашем сообщении даёт `3B 1F 2D 32` и не больше 3 нулей подряд, но это свойство именно этого сообщения. Серия единиц любой длины превращается в такую же серию нулей, поэтому в общем случае инверсия вредна. Выборочная инверсия с флагом (DBI в DDR4) — уже избыточное кодирование.

5.3. Сравнение

	4B/5B	**Скремблирование**
Избыточность	25 %, линия 12,5 Мбит/с, спектр шире	нет, скорость в линии не меняется
Гарантия	не больше 3 нулей подряд для любых данных	нет, зависит от данных

Ошибки	запрещённые коды обнаруживаются	не обнаруживает, размножает на 3 бита
Сложность	таблица перекодировки	регистр сдвига и XOR, нужна синхронизация

6. Ваш отчёт: числа и что говорить

6.1. Сообщение и физические коды (стр. 3–8)

- Сообщение ДаТН: **Да** — первые две буквы фамилии Данилов, **Т** — Тимофей, **Н** — Николаевич. Коды по таблице задания: Д C4, а E0, Т D2, Н CD.
- 32 бита, 15 единиц и 17 нулей. Самая длинная серия нулей — 5 (на стыке E0 и D2), серия единиц — 3.
- Номер в списке 6 (чётный) $\rightarrow C = 10$ Мбит/с.

Код	$f_{н}$, МГц	$f_{в}$, МГц	$f_{ср}$, МГц	S, МГц	Участки
NRZ	1	5	3,34	4	17 полуволн
RZ	5	10	7,42	5	31 стык: 15 по C, 16 по $C/2$
AMI	1,43	5	2,69	3,57	15 импульсов
MLT-3	0,91	1,67	1,19	0,76	8 импульсов
Манчестер	5	10	8,33	5	48 полуволн: 32 по C, 16 по $C/2$

Почему именно **Манчестер и MLT-3** выбраны как два лучших: Манчестер — самосинхронизация на любых данных, нет постоянной составляющей, два уровня. MLT-3 — самая узкая полоса. Недостаток MLT-3 (серии нулей) убирается логическим кодированием, поэтому этапы 3–4 сделаны для него.

6.2. Логическое кодирование и итог (стр. 8–13)

Способ	Бит	Избыт.	Линия, Мбит/с	Макс. серия нулей	$f_{н}$	$f_{в}$	$f_{ср}$	S
Манчестер	32	0	10	5	5	10	8,33	5
MLT-3	32	0	10	5	0,91	1,67	1,19	0,76
4B/5B + MLT-3	40	25 %	12,5	2	1,25	3,13	2,06	1,88
Скремблирование (5/7) + MLT-3	32	0	10	4	0,83	2,5	1,61	1,67
Инверсия + MLT-3	32	0	10	3	1	2,5	1,51	1,5

Вывод отчёта: лучший способ — **4B/5B + MLT-3**: полоса 1,25…3,13 МГц, гарантированно короткие серии нулей, обнаружение ошибок, синхронизация. Цена — избыточность 25 % и линия 12,5 Мбит/с. Такое сочетание используется в Fast Ethernet 100BASE-TX.

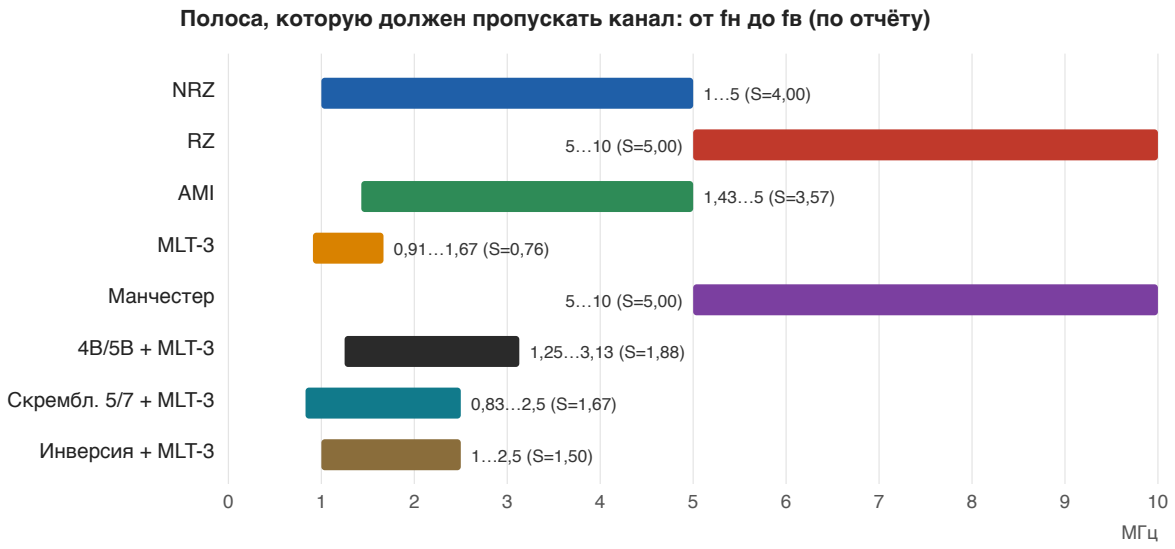


Рисунок 6. Требуемая полоса канала для каждого способа (значения из отчёта)

Почему выбран полином 5/7, а не 3/5: он поднимает число единиц с 15 до 19 (больше смен уровня в MLT-3) и даёт $f_н = 0,83$ МГц против 0,63 МГц у 3/5. Честная оговорка из отчёта: у 3/5 серия единиц короче (2 против 5), а $f_н$ у 5/7 всё равно ниже, чем у исходного MLT-3 (0,91). Поэтому скремблирование на таком коротком сообщении почти не помогло.

Раздел 6.1 (таблица 15): на разных входах (текст, чередование 0101…, потоки нулей и единиц) инверсия и скремблер не гарантируют результата; только 4B/5B на любом входе даёт не больше трёх нулей подряд. Перебор 132 скремблеров (приложение Б) не нашёл варианта, лучшего на всех обычных данных. Причина: скремблер без избыточности — перестановка сообщений, в среднем не лучше случайных бит.

7. Слабое место отчёта: краевые импульсы MLT-3

Что обнаружено. Я независимо пересчитал таблицы 5, 9 и 12. Они совпадают с отчётом только если у **первого и последнего импульса** соседний нулевой участок берётся **целиком**, а не пополам. Остальные импульсы считаются по правилу «пополам» (пример в отчёте: $3 + 2 + 0,5 = 5,5\tau$). Правило «пополам» в тексте на стр. 3 сформулировано без оговорки про края.

Пример на MLT-3 сообщения: первый импульс (1 бит) за которым 4 нуля. Если делить пополам: $1 + 2 = 3\tau$. В отчёте: $1 + 4 = 5\tau$. Последний импульс (1 бит) после 2 нулей: пополам $1 + 1 = 2\tau$, в отчёте $1 + 2 = 3\tau$.

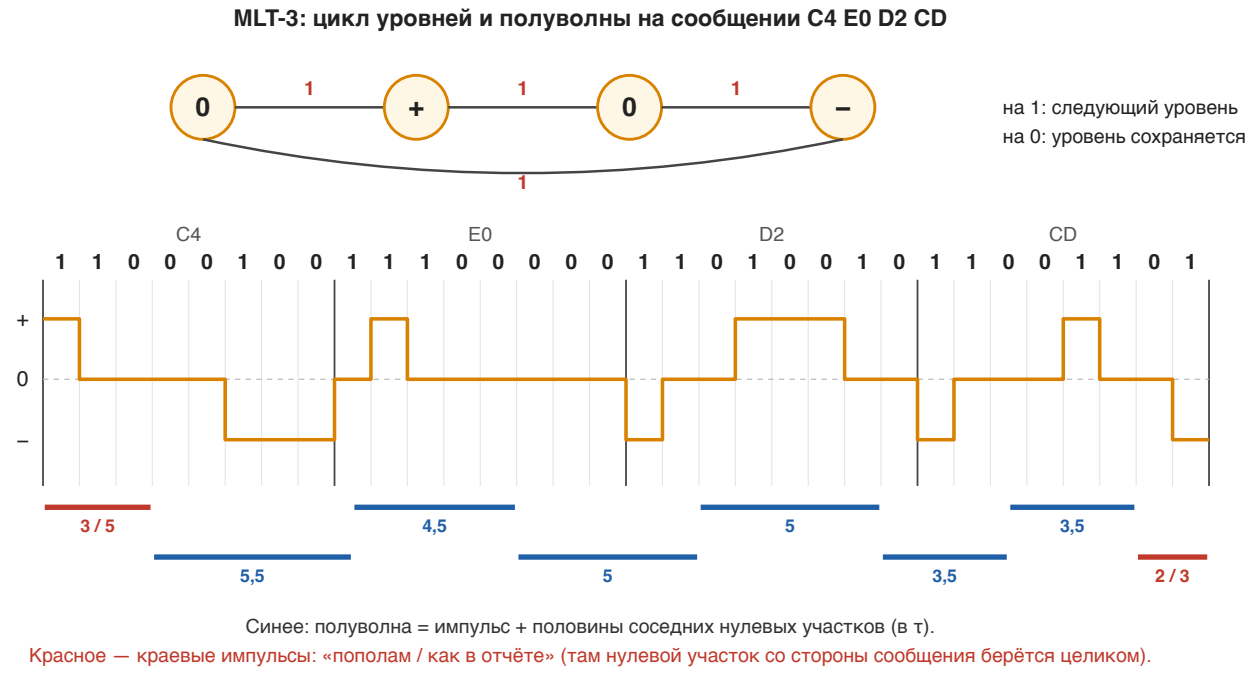


Рисунок 7. Автомат MLT-3 и полуволны на вашем сообщении. Красным — краевые импульсы, где метод «пополам» и метод отчёта расходятся

Что меняется, если считать везде пополам (мой пересчёт, не из отчёта):

Способ	$f_н$ отчёт	$f_н$ пополам	$f_в$ отчёт	$f_в$ пополам	$f_{ср}$ пополам	S пополам
MLT-3	0,91	0,91	1,67	2,5	1,38	1,59
4B/5B + MLT-3	1,25	1,25	3,13	4,17	2,21	2,92
Скремб. 5/7	0,83	1,11	2,5	2,5	1,68	1,39
Скремб. 3/5	0,63	0,62	—	3,33	1,60	2,71
Инверсия	1	1	2,5	2,5	1,56 (отчёт 1,51)	1,5

Что это значит для выводов:

- Выбор лучшего способа (4B/5B + MLT-3) не меняется: $f_н$ у него по-прежнему самая высокая (1,25), серия нулей самая короткая.
- Но фразы «верхняя граница в 6 раз ниже, чем у манчестерского» и «в 3,2 раза» становятся «в 4 раза» и «в 2,4 раза».
- Для скремблера 5/7 нижняя граница по правилу «пополам» была бы 1,11, то есть выше исходных 0,91, и вывод «скремблер даже ухудшил $f_н$» не подтверждается.

Что говорить, если спросят «почему первый и последний импульс считаешь иначе?»:

1. Честный вариант: «У краевого импульса с внешней стороны нет соседа, с которым делить нулевой участок, поэтому я отнёс участок целиком к краевому импульсу. Если делить пополам, $f_в$ MLT-3 равна 2,5 МГц, выводы о выборе кода не меняются, меняются только отношения «в N раз ниже».»
2. Заранее откройте свой `compute.py` и проверьте, как там реализованы края. Если там ошибка, скажите об этом сами до того, как её найдёт преподаватель.
3. Не утверждайте, что «правило всегда пополам»: это противоречит вашим же таблицам.

Дополнительно: README папки `study/telecom` пишет номер в списке 2, отчёт — 6. Оба чётные ($C = 10$), но сверьтесь со списком в ИСУ.

8. Вопросы на защиту с ответами

1. **Что такое потенциальное кодирование?** Бит задаётся уровнем на всём битовом интервале (NRZ: 1 — высокий, 0 — низкий).

2. **Чем бод отличается от бит/с?** Бит/с — информационная скорость, бод — число символов (состояний) в секунду. У NRZ, AMI, MLT-3 равны; у манчестера и RZ бод $= 2C$; у PAM-5 бод $= C/2$.
3. **Чем плоха длинная серия нулей или единиц в NRZ?** Постоянный уровень: нет смен для синхронизации, появляется постоянная составляющая, $f_{\text{н}} \to 0$, спектр расширяется вниз.
4. **Когда нет постоянной составляющей?** Когда время на положительном и отрицательном уровнях одинаково: манчестер (всегда), AMI на единицах, MLT-3 в среднем, NRZ при чередовании 1 и 0.
5. **Почему в телефонных каналах (300–3400 Гц) нельзя NRZ?** У кода спектр начинается около 0 Гц, а канал не пропускает постоянный ток и низкие частоты. Используют модуляцию.
6. **Импульсные и потенциальные коды?** Потенциальные — уровнем, узкий спектр, но нужна синхронизация. Импульсные — перепадом/импульсом, шире спектр, но самосинхронизация.
7. **Какой код имеет наименьшую верхнюю границу?** PAM-5 ($C/4$ по символу). Из рассмотренных в работе — MLT-3 ($C/4$ на длинной серии единиц; в этом сообщении 1,67 МГц).
8. **Зачем логическое кодирование?** Убрать длинные серии 0/1: даёт самосинхронизацию, убирает постоянную составляющую, поднимает $f_{\text{н}}$, у избыточных кодов ещё и обнаруживает ошибки.
9. **Методы логического кодирования?** Избыточное (4B/5B, 5B/6B, 8B/10B, 8B/6T) и скремблирование.
10. **Что такое запрещённые коды?** Комбинации, не соответствующие ни одному исходному символу. Появление на приёме = ошибка.
11. **Избыточность 4B/5B?** $(5-4)/4 = 25\ \%$: на 32 бита данных 40 бит в линии, то есть 8 лишних.
12. **Почему линия 12,5 Мбит/с?** Чтобы полезная скорость осталась 10 Мбит/с, на 32 бита данных нужно передать 40: $C' = 1,25 \cdot 10$.
13. **Почему при 4B/5B выросла $f_{\text{в}}$?** Линия работает на $C' = 12,5$, а все частоты пропорциональны скорости.
14. **Почему 4B/5B делали с MLT-3, а не с манчестером?** Манчестер и так самосинхронизируется, логическое кодирование ему не нужно и только расширит спектр. Оно нужно потенциальным кодам.
15. **Как работает дескремблер?** $C_i = B_i \oplus B_{i-5} \oplus B_{i-7} = A_i$, потому что XOR одного и того же значения дважды возвращает исходный бит.
16. **Что будет при одной ошибке в канале у скремблера?** Размножение: испортятся биты i, $i+5$, $i+7$ (три бита).
17. **Почему скремблирование не лучшее решение?** Ничего не гарантирует и не обнаруживает ошибок; на вашем сообщении серия нулей сократилась только до 4.
18. **Почему в вашем сообщении инверсия «работает», а в общем случае нет?** Свойство именно этих данных: в нём мало единиц подряд. Серия единиц превращается в серию нулей.
19. **Что такое 100BASE-TX?** Fast Ethernet: 4B/5B + MLT-3, линия 125 Мбод, основная гармоника не выше 31,25 МГц.
20. **Как найти $f_{\text{в}}$ и $f_{\text{н}}$ по диаграмме?** Найти самую короткую и самую длинную полуволну, подставить в $f = C/(2L)$.
21. **Почему $F \geq S$?** Канал должен пропускать весь диапазон основных гармоник сигнала, иначе часть информации будет потеряна.
22. **Зачем 3-я гармоника?** Прямоугольный сигнал точнее воспроизводится, если канал пропускает не только f_0, но и $3f_0$ (амплитуда $1/3$). Полоса тогда втрое больше.
23. **Почему самая длинная серия нулей в MLT-3 важна?** На нулях уровень не меняется, нет смен для синхронизации и $f_{\text{н}}$ уменьшается.
24. **Как работает AMI и что даёт?** Единицы чередуют $+$ и $-$; нарушение чередования = ошибка; на единицах нет постоянной составляющей.

9. Шпаргалка

Формулы. $\tau = 1/C$ (100 нс). $f = C/(2L)$. $S = f_{\text{в}} - f_{\text{н}}$. $f_{\text{ср}} = \sum f_i n_i / \sum n_i$. $F \geq S$.
Сообщение. `C4 E0 D2 CD`, 32 бита, 15 единиц, 5 нулей подряд на E0|D2.
NRZ. $f_{\text{н}} = 1$, $f_{\text{в}} = 5$, $S = 4$, $f_{\text{ср}} = 3,34$.
Манчестер / RZ. $f_{\text{н}} = 5$, $f_{\text{в}} = 10$, $S = 5$.
AMI. 1,43…5, $S = 3,57$. **MLT-3.** 0,91…1,67, $S = 0,76$ (см. раздел 6 про края).
4B/5B. `D2 B9 ED D3 5B`, 40 бит, 25 %, $C' = 12,5$, 1,25…3,13 МГц, $S = 1,88$; не больше 3 нулей подряд.
Скремблер. $B_i = A_i \oplus B_{i-5} \oplus B_{i-7}$, `C3 7D C7 78`; дескремблер той же формулой на принятых битах; размножает ошибку на 3 бита.
Инверсия. `3B 1F 2D 32`, работает только на этих данных.
Итог. Лучший способ — 4B/5B + MLT-3 (как в 100BASE-TX).
Таблица 4B/5B. 0:11110 1:01001 2:10100 3:10101 4:01010 5:01011 6:01110 7:01111 8:10010 9:10011 A:10110 B:10111 C:11010 D:11011 E:11100 F:11101.